

Gestion de Desarrollo de Software

Guia de Estudio Detallada - Parcial 1

TUP - Gestion de Desarrollo de Software - Parcial 1

Basado en U1 Intro, U2 Propuesta, U3 Metricas, Metodologia SI y Pruebas

Agosto 2026

Gestión de Desarrollo de Software - Guía de Estudio Completa P1

TUP - Parcial 1 | Basado en: U1 Intro, U2 Propuesta, U3 Métricas, Metodología SI, Pruebas Caja Blanca/Negra

Última actualización: Agosto 2026

Índice

1. Mapa Conceptual General
2. Las 4P de la Gestión (Pressman)
3. Producto: Qué es y cómo se deteriora
4. Proceso: Marco de Trabajo y Modelos de Ciclo de Vida
5. Proyecto: Definición, Planificación y Riesgo
6. Roles en el Desarrollo
7. Requerimientos
8. Propuesta de Proyecto
9. Alcance del Proyecto
10. Stakeholders y Matriz Poder/Interés
11. Suposiciones, Restricciones y Criterios de Aceptación
12. Métricas y Estimaciones
13. Metodología de Sistemas de Información (10 Etapas)
14. Pruebas: Estrategia, Caja Blanca y Caja Negra
15. Anexo A: Tablas Resumen para Memorizar
16. Anexo B: Glosario Clave
17. Cómo Estudiar con esta Guía

1. Mapa Conceptual General

TEORÍA GENERAL DE SISTEMAS (Pensamiento Lineal + Sistémico)

|
?

METODOLOGÍA DE SISTEMAS DE INFORMACIÓN (10 etapas macro)

| Reconocimiento -> ... -> Factibilidad -> [elige solución]

|

|

|

?-----?

|

?

|

INGENIERÍA DE SOFTWARE (dentro de cada proyecto de desarrollo)

|

| 4P: Personas-Producto-Proceso-Proyecto

|

| - Propuesta (U2) = contrato inicial

|

| - Métricas/Estimación (U3) = ¿cuánto cuesta/cuánto tarda?

|

| - Pruebas (Caja Blanca/Negra) = calidad

?

MANTENIMIENTO / SUSTITUCIÓN -> nuevo ciclo

Idea clave para el parcial: La Metodología SI es el contenedor; la Ingeniería de Software es un proyecto dentro de ella (cuando la alternativa solución es construir software). No confundir etapas de MSI con actividades del proceso de software.

2. Las 4P de la Gestión

Según Pressman, la administración efectiva se enfoca en 4 pilares interdependientes.

2.1 Personas

Todo proyecto está poblado de participantes: Gerentes Ejecutivos, Gerentes de Proyecto, Profesionales (técnicos), Clientes, Usuarios Finales.

Líder de Proyecto ideal - 3 capacidades:

1. Motivación: alentar al personal a producir a su máxima capacidad.
2. Organización: moldear procesos existentes o inventar nuevos para traducir concepto en producto.
3. Ideas/Innovación: fomentar creatividad aun dentro de fronteras del producto.

Equipo de alto rendimiento requiere:

- Confianza mutua.
- Distribución de habilidades adecuada al problema.
- Capacidad de excluir inconformes para mantener cohesión (polémico pero citado por Pressman).

Concepto "Equipo cuajado" (DeMarco/Lister): El todo es mayor que la suma de las partes. No todo grupo asignado a trabajar junto es un equipo.

Toxicidad de equipo - 5 factores (PREGUNTA FRECUENTE):

1. Atmósfera de trabajo frenético.
2. Alta frustración ? fricción entre miembros.
3. Proceso de software fragmentado o pobremente coordinado.
4. Definición poco clara de roles.
5. Continua exposición al fracaso.

2.2 Producto, Proceso, Proyecto

Se desarrollan en capítulos siguientes.

3. Producto

¿Qué es? No es solo código. Es: programas + ejecutables + código fuente + diseño lógico de datos + documentación (especificaciones, modelos, diagramas, manuales).

¿Quién lo hace? Ing. en Sistemas, Arquitectos, Diseñadores, Analistas, Programadores.

¿Pasos para obtenerlo? Definidos por la metodología elegida (lineal, incremental, iterativo, ágil).

3.1 El Producto se Deteriora (NO se desgasta)

Esta comparación Hardware vs Software es central para justificar mantenimiento.

Característica | Hardware | Software

Curva fallas temprana | Alta (defectos fabricación) -> se corrige -> meseta estable | Alta (defectos ocultos) -> se corrige -> meseta

Desgaste físico | Sí: suciedad, vibración, temperatura -> tasa fallas vuelve a subir | NO: no sufre ambiente

Curva ideal | Baja, sube al final por desgaste | Plana tras corrección inicial

Curva REAL | - | Se deteriora: cada cambio/mantenimiento introduce nuevos errores -> picos, y el nivel base de fallas sube gradualmente. El software se vuelve obsoleto si no se ...

Conclusión examen: Software no se desgasta, pero Sí se deteriora con los cambios. Si no se diseña para mantenibilidad, cada corrección empeora la curva.

4. Proceso

Definición: Estructura para actividades, acciones y tareas para construir software de alta calidad.

- Actividad: objetivo amplio (ej: Comunicación).
- Acción: conjunto de tareas que produce producto de trabajo.
- Tarea: objetivo pequeño y bien definido (ej: prueba unitaria).

4.1 Marco Genérico (5 Actividades Estructurales + Sombrilla)

Estructurales (siempre, en algún orden):

1. Comunicación
2. Planeación
3. Modelado
4. Construcción
5. Despliegue

Sombrilla / Custodiales (a lo largo de TODO el proceso):

Seguimiento y control, Administración de riesgos, Aseguramiento de calidad (SQA), Administración de configuración, Revisiones técnicas.

Cada actividad estructural = conjunto de acciones = conjunto de tareas (con productos, QA points y milestones).

4.2 Modelos de Ciclo de Vida - Tabla Maestra

Modelo | Tipo | Flujo | Participación Cliente | Entrega | Riesgo | División trabajo | Paralelismo

Cascada (Waterfall) | Lineal | Secuencial: Req -> Análisis -> Diseño -> Codificación -> Prueba -> Implementación | Muy escasa (inicio y final) | Ejecutable solo al final | Detec...

Incremental | Incremental | Secuencial por incrementos de funcionalidad (sin refinamiento) | Escasa por incremento | Ejecutable al final de cada incremento | Medio | Sí | No

Espiral | Iterativo/Evolutivo | Cíclico, crece definición y baja riesgo por vuelta, usa prototipos | Alta retroalimentación | Versión refinada cada iteración | Gestiona riesgo e...

Proceso Unificado (PU) | Iterativo e Incremental | Mini-proyectos 2-6 sem, dirigido por Casos de Uso, centrado en Arquitectura | Alta | Ejecutable integrado cada iteración | Med...

Ágil (Scrum, Kanban, XP) | Iterativo e Incremental | Sprints 1-4 sem, equipos autoorganizados, valor al cliente | Continua | Software funcional cada sprint | Adaptativo | No ríg...

"Proceso Madre": Cascada, porque sus pasos genéricos aplican a todos los demás.

Manifiesto Ágil (4 valores) - MEMORIZAR:

1. Individuos e interacciones sobre procesos y herramientas
2. Software funcionando sobre documentación extensiva
3. Colaboración con cliente sobre negociación contractual
4. Responder al cambio sobre seguir un plan

Aunque lo de la derecha tiene valor, se valora más lo de la izquierda.

4.3 Clasificación Predictivo vs Adaptativo

Enfoque | Conocimiento Requisitos | Planificación | Entregas | Ejemplo del material

Predictivo | Total / casi total | Completa al inicio, única entrega final | 1 entrega | Construcción de casa

Iterativo | Incertidumbre media | Entrega evolucionada no definitiva, se refina hasta entrega única final | Evolutivas, perfeccionamiento | Construcción de robot

Incremental | Incertidumbre media | Entrega funcional real cada iteración, se adiciona y queda usable | Incrementales usables | -

Ágil | Mucha incertidumbre | Iterativo + Incremental: entrega funcional real + retroalimentación | Funcional cada sprint | -

4.4 ¿Qué modelo elegir? (Cuadro del parcial U1:25)

- App Educativa (feedback usuarios, contenido evolutivo, actualizaciones frecuentes) ? Ágil/Iterativo
- Sistema Bancario (seguridad, precisión transacciones, integridad datos, errores catastróficos) ? Cascada/Predictivo o Espiral (formal, trazable)
- Sistema Comercial con Login (incertidumbre mercado, validar rápido, recursos limitados) ? Ágil Incremental
- Control Acceso Escolar (seguridad, integración, workflows definidos, normativa) ? Incremental o Cascada

5. Proyecto

Definición: Esfuerzo temporal para crear producto/servicio/resultado único.

5 Características:

1. Alcance y entregables definidos (criterios éxito)
2. Cronograma fijo (inicio-fin ? urgencia)
3. Recursos asignados (humanos, financieros, técnicos) limitados
4. Expectativas calidad / criterios aceptación
5. Gestión activa (planificación, seguimiento, control)

Registrar antecedentes: Historia para mejorar estimaciones futuras.

5.1 Proyecto vs Tarea Rutinaria

Característica | Proyecto | Tarea Rutinaria

Duración | Temporal, fin definido | Continua, repetitiva

Singularidad | Crea resultado único | Produce similares

Planificación | Exhaustiva | Procedimientos establecidos
 Incertidumbre | Mayor riesgo, cambios frecuentes | Predecible, estable
 Recursos | Asignación variable | Consistente
 Gestión | Gestión de proyectos | Gestión operativa

Ejemplos Proyecto: App móvil festival, nuevo CRM, plataforma e-learning universidad, rediseño web, inventario minorista.

Ejemplos Rutina: Facturación mensual, backup diario, corrección bugs semanal, mantenimiento regular, atención al cliente continua.

5.2 Planificación del Proyecto

Objetivo: Marco para estimar recursos, costo y calendario con escenarios mejor/peor caso para acotar resultados.

5 Tareas:

1. Establecer ámbito y factibilidad
2. Analizar riesgos
3. Definir recursos (personas-roles, componentes reutilizables, entorno HW/SW)
4. Estimar costo y esfuerzo
5. Desarrollar calendario

6 Señales de peligro (proyecto en riesgo):

1. Personal no entiende necesidades cliente
2. Ámbito pobremente definido
3. Cambios gestionados pobremente
4. Fechas límite irreales
5. Usuarios resistentes
6. Equipo sin habilidades adecuadas

Caso debate: Planificado (req claros, doc, revisiones, pruebas sistemáticas) vs Improvisado (verbal, doc mínima, ad hoc, reactivo) ? el planificado tiene más prob. éxito en calidad/cronograma/presupuesto/satisfacción, aunque improvisado puede ser rápido en contextos muy pequeños pero no escalable.

5.3 Administración del Riesgo

Riesgo = problema potencial. 2 características: Incertidumbre (puede o no ocurrir, nunca 100% prob.) + Pérdida (consecuencias si ocurre). Cuantificar ambas.

Estrategias:

- Reactiva: "No te preocupes, pensaré en algo" cuando sucede. (Mala)
- Proactiva: Identificar ? valorar prob/impacto ? clasificar ? plan mitigación/monitoreo/contingencia. (Inteligente)

3 Categorías:

- Proyecto: presupuesto, personal, recursos, cronograma
- Técnico: diseño, implementación, interfaz, mantenimiento
- Empresarial: mercado, estrategia (ej: producto ya no requerido)

3 Tipos según conocimiento:

- Conocidos: descubribles evaluando plan (fecha irreal, requisitos no documentados)
- Predecibles: experiencia previa (rotación personal, pobre comunicación cliente)
- Impredicibles: muy difíciles de anticipar

3 Ejes de gestión proactiva: 1) Evitar, 2) Monitorear, 3) Manejar + plan contingencia.

Tabla de Riesgos (técnica simple de proyección):

Riesgo | Categoría | Probabilidad | Impacto (1 catastrófico - 4 despreciable)

Alta rotación personal | Proyecto | 70% | 2 Crítico

Cambio requisitos cliente | Proyecto | 60% | 2

... | ... | ... | ...

Pasos: Estimar prob. (consenso equipo) + impacto ? ordenar descendente por prob*impacto ? línea de corte ? solo riesgos sobre la línea llevan plan de mitigación/monitoreo/manejo (RMMM).

Ejemplo completo - Rotación personal:

- Mitigación: reunirse para causas, mejorar condiciones, suponer que ocurrirá, dispersar conocimiento (equipos cruzados), estándares y docs oportunas, revisiones pares, asignar respaldo por cada técnico crítico.
- Monitoreo: actitud equipo bajo presión, cohesión, relaciones interpersonales, compensación, ofertas externas.

- Manejo/Contingencia: si ocurre, respaldo disponible, conocimiento disperso, docs al día.

6. Roles

El desarrollo es esfuerzo de equipo; definición clara evita superposición.

Rol | Responsabilidades Principales | Habilidades | Actividades Diarias

Desarrollador	Código limpio/mantenible, implementar funciones, traducir diseño, pruebas unitarias	Lenguajes, resolución problemas, arquitectura, control versiones	Codifica...
Evaluador / QA	Diseñar/ejecutar casos prueba, documentar bugs, verificar correcciones, pruebas regresión	Técnicas prueba, atención detalle	Pruebas funcionales, rendimiento...
Analista Funcional	Recopilar necesidades (entrevistas/talleres), traducir a requisitos, puente negocio-técnico, documentar specs	Comunicación, modelado, pensamiento sistémico...	
Líder Proyecto	Coordinar, asignar recursos, cronograma/hitos, monitorear progreso, gestión riesgos, reportar, resolver conflictos	Técnica + gestión personas	Planning, segu...
Usuario/Cliente	Definir necesidad/ contexto, feedback prototipos, validar solución, aceptación final	Conocimiento dominio	Revisar, sugerir ajustes, aprobar

Roles Ágiles (responsabilidad compartida, no rígidos):

- Product Owner: Maximiza valor, prioriza backlog, representa negocio.
- Scrum Master: Facilita proceso, elimina impedimentos, asegura prácticas ágiles.
- Equipo Ágil: Multifuncional, autoorganizado, entrega continua, progreso diario.

7. Requerimientos

"Los requisitos son la base sobre la que se construye el software. Si la base es débil, fallará aunque el diseño sea bueno."

- Funcional (QUÉ hace): características, comportamientos, E/S. Ej: "Permitir registro con email y contraseña" / "Generar informes mensuales".
- No Funcional (CÓMO lo hace): restricción sobre funcional. Rendimiento, seguridad, usabilidad, confiabilidad. Ej: "Registro en <2s" / "Compatible móviles" / "Cumplir RGPD" / "Accesible discapacitados visuales".

Tabla ejemplos clave:

Funcional | No Funcional asociado

Registro con email/pass | Completarse en 2s
 Enviar email confirmación | Compatible todos los móviles
 Restablecer contraseña | Cumplir RGPD
 Generar informes asistencia | Accesible visuales
 Marcar presente/ausente 1 acción | Accesible pantallas 4 pulgadas

4 Errores comunes (EVITAR):

1. Ambigüedad - lenguaje vago: "responder rápidamente" ? ¿1s o 5s? Definir métrica.
2. Suposiciones no validadas: asumir internet alta velocidad o Excel instalado ? prever alternativas (PDF).
3. Cambios no documentados: acuerdos verbales sin actualizar docs ? scope creep.
4. Combinar requisitos: "Login debe usar JWT y verse profesional" ? separar en 2: técnico (JWT) + UX (guía marca).

De objetivos ? requisitos: Consultar cliente, indagar síntomas/causas, pensamiento sistémico. Responsabilidad compartida profesional-usuario.

4 Técnicas de relevamiento:

1. Entrevistas partes interesadas (individual/grupal)
2. Cuestionarios (estructurados, útil distribuido)
3. Análisis sistema actual (fortalezas/debilidades/oportunidades)
4. Prototipos (validación temprana visual)

8. Propuesta de Proyecto

Definición: Documento fundamental, brújula inicial, comunicación desarrolladores ? stakeholders. Concisa pero completa: demuestra viabilidad sin ahogar en detalle técnico (eso viene después).

¿Por qué importa? Alinea dirección, crea entendimiento compartido, permite evaluar viabilidad técnica/recursos antes de invertir.

8 Componentes Esenciales

Componente | Pregunta que responde | Tip

1. Descripción general | ¿Qué construimos? Título memorable + descripción concisa | Indicativo de solución
 2. Declaración problema | ¿Qué dolor resuelve? Con datos cuantitativos si posible | Demostrar impacto
 3. Objetivos | ¿Qué metas específicas/medibles lograremos? | SMART
 4. Definición alcance | ¿Qué incluye/excluye? (ver cap. 9) | Marco inicial para detalle posterior
 5. Partes interesadas | ¿Quién está involucrado/afectado? | Lista completa (ver cap. 10)
 6. Suposiciones/Restricciones | ¿Qué asumimos y qué límites fijos tenemos? | -
 7. Criterios aceptación | ¿Cómo sabemos que es "hecho" y aprobado? | Objetivos y medibles
- + Presentación | Documento + exposición verbal (ver 11.3) | Complementarios

9. Alcance del Proyecto

Define límites: qué se entrega y qué NO. Es entendimiento contractual. Previene malentendidos, establece cuándo está completo (evita alcance infinito), permite planificar y negociar concesiones.

Inclusiones: funcionalidades/compromisos concretos. Expresar en capacidades de usuario (ej: "Usuarios pueden registrarse con verificación email" mejor que "Implementa OAuth").

Exclusiones: tan importantes como inclusiones. Sin ellas, stakeholders asumen funciones "por defecto". Ser explícito.

Ejemplo material:

Inclusiones | Exclusiones

Registro y gestión perfiles | Apps móviles iOS/Android
 Informes básicos export PDF | Integración pagos terceros
 Panel admin gestión usuarios | Análisis avanzado
 Backup y restauración | Notificaciones SMS/WhatsApp automáticas
 Validación formularios | Multilenguaje

Tabla de definición de alcance (formato recomendado para entrega):

Funcionalidad | Incluida | Excluida | Observaciones

Registro usuario | Verificación email | - | Incluida
 Informes | PDF plantillas básicas | Avanzados | -
 Notificaciones | - | WhatsApp | Posible mejora futura
 Backups | - | Automatizadas | Manual documentado
 Pagos | - | Mercado Pago | v2.0
 Idioma | - | Multi-idioma | Solo español
 Admin | Métricas básicas + gestión | - | -

10. Stakeholders

Definición: Personas/grupos/orgs con interés, influencia o afectadas por el proyecto (dentro y fuera).

Características: Diferentes niveles influencia/interés, prioridades contrapuestas, impacto directo en éxito, requieren comunicación diferenciada.

5 Clasificaciones:

1. Tomadores decisión: aprueban presupuesto/cronograma (Director financiador)
2. Usuarios finales: usan software diario (Estudiantes)
3. Equipo desarrollo: construye (Devs, QA)
4. Autoridades académicas: supervisan/evalúan (Profesor)
5. Indirectas: afectadas indirectamente (Soporte IT que mantendrá)

¿Por qué identificarlas? Define canales/frecuencia comunicación, captura requisitos ocultos, anticipa conflictos.

10.1 Matriz Poder/Interés (PREGUNTA SEGURA DE PARCIAL)

| Bajo Poder | Alto Poder

Bajo Interés | Monitorear (comunicación mínima, ocasional) | Mantener satisfecho (actualizaciones intervalos adecuados, sin sobrecargar)

Alto Interés | Mantener informado (comunicación regular, alentar participación) | Gestionar de cerca (compromiso total, info detallada regular)

- Poder: capacidad influir decisiones/recursos.
- Interés: cuánto les importa resultado / grado participación.

Ejemplo Sistema Registro Cursos:

- Directores académicos: Alto Poder/Alto Interés ? Gestionar de cerca (informes regulares, consulta decisiones clave). Preocupa: fiabilidad, seguridad datos, cumplimiento política.
- Soporte IT: Poder medio/Interés medio ? entre "Mantener informado" y "Satisfecho", involucrar en decisiones técnicas/capacitación. Preocupa: soporte, docs, integración.
- Estudiantes: Bajo Poder/Alto Interés ? Mantener informado (updates, beta, feedback). Preocupa: usabilidad, móvil, velocidad.

11. Suposiciones, Restricciones y Criterios

11.1 Suposiciones vs Restricciones

| Suposiciones | Restricciones

Naturaleza | Se consideran verdaderas para planificar, no garantizadas, con incertidumbre | Limitaciones fijas conocidas desde inicio

Manejo | Requieren gestión riesgo + planes contingencia y validación | No negociables, hay que acomodarse, dan forma a decisiones

Origen | Factores externos fuera control | Internas/externas pero definidas

Ejemplos:

Suposiciones | Restricciones

Usuarios tienen conocimientos básicos informática | Entrega final en 10 semanas

Cliente proporcionará datos a tiempo | Solo tecnologías open source

Servidores ya disponibles | Equipo limitado a 3 devs

APIs terceros mantendrán compatibilidad | Presupuesto \$50.000

Requisitos permanecerán estables | Cumplir RGPD

11.2 Criterios de Aceptación

Condiciones específicas para que cliente apruebe = contrato claro de "hecho". Eliminan ambigüedad, aseguran calidad, satisfacen cliente, protegen legalmente.

Ejemplos medibles:

- Funcional: "Procesar correctamente todas transacciones inventario según reglas v2.3"
- Rendimiento: "Carga página <2s con 100 usuarios simultáneos"
- Usabilidad: "Nuevos usuarios completan flujo principal sin capacitación"
- Compatibilidad: "Funciona en principales móviles/navegadores últimos 3 años"

11.3 Presentación (Verbal + Escrita)

Ambos formatos complementarios; escrito con más detalle.

Documento escrito: simple, bien organizado, encabezados claros, formato consistente.

Verbal: breve, lenguaje claro, flujo lógico, tiempo equilibrado entre miembros.

Tips: claridad/brevedad (evitar jerga o explicarla), participación todo equipo con transiciones practicadas, proyectar confianza (dominar tema, no leer notas), lenguaje sencillo.

¿Por qué practicar dentro del equipo? Comunica visión, recibe feedback interno, practica habilidad profesional, simula real, detecta lagunas planificación.

12. Métricas y Estimaciones

¿Por qué estimar? Anticipar esfuerzo/duración, crear cronogramas realistas (hitos), establecer expectativas transparentes con cliente.

Principios clave:

- Son aproximaciones fundadas, no predicciones exactas ni adivinación.
- Marco razonable, contienen incertidumbre inherente ? objetivo reducir rango a manejable.
- Mejoran con experiencia y datos históricos.

Consecuencias mala estimación:

Subestimación | Sobreestimación

Cronogramas comprimidos, problemas calidad, burnout, recursos emergencia caros, posible fracaso | Presupuesto inflado, insatisfacción cliente, oportunidades perdidas, scope cree...

12.1 Tres Técnicas

1. Juicio de Expertos

- Basado en experiencia personal de seniors/managers en proyectos similares.
- Rápida, mínimo formalismo, eficaz si hay amplia experiencia, conocimiento tácito.
- Formalizable: entrevistas estructuradas o Técnica Delphi ? rondas anónimas con cuestionarios ? análisis ? informe feedback ? segundo cuestionario ? repetir hasta consenso.

2. Estimación por Analogía

- Compara proyecto actual con pasados similares usando métricas históricas.
- Ideal si hay métricas detalladas y complejidad comparable. Depende de similitud alcance/tecnología/equipo.

3. Descomposición de Tareas

- Divide proyecto grande en componentes pequeños manejables ? estima cada uno ? suma + contingencia integración.
- Mejora precisión al reducir alcance por unidad.

Técnicas de descomposición: LOC (Líneas Código), PF (Puntos Función), PCU (Puntos Casos Uso).

- LOC: requiere descomposición muy detallada; a mayor partición, mayor precisión.
- PF: estima características dominio info (entradas, salidas, archivos, consultas, interfaces externas).

Para LOC y PF: estimar con histórico o intuición (si falla todo) 3 valores: Optimista (O), Más probable (M), Pesimista (P) ? Fórmula PERT:

$$VE = (O + 4M + P) / 6 \text{ (promedio ponderado, da 4x peso a M)*}$$

Ventajas PERT: considera incertidumbre estructurada, reduce impacto opiniones extremas, enfoque ponderado por probabilidad.

Ejemplo Login:

O=4h (experto, sin integración), M=8h (normal con debug), P=20h (seguridad + API + cambios cliente)

$$? VE = (4 + 4 \cdot 8 + 20) / 6 = (4 + 32 + 20) / 6 = 56 / 6 = 9.33h$$

12.2 Fórmula Costos

$$\text{Costo Total} = (\text{Esfuerzo Estimado} \times \text{Tarifa por Hora}) + \text{Costos Adicionales}$$

- Esfuerzo: horas-hombre (tras PERT)
- Tarifa: por perfil
- Junior (0-2 años, tareas simples con supervisión): \$300/h
- Semi Senior (2-5 años, independiente funciones estándar): \$500/h
- Senior (5+ años, diseña, mentorea, problemas complejos): \$800/h
- Adicionales: licencias, infraestructura (hosting/cloud/BD), documentación, mantenimiento.

Planilla ejemplo material:

Módulo	Hs PERT	Perfil	Tarifa	Subtotal
Registro Usuarios	10	Junior	\$300	\$3.000
Generación Informes	15	Semi Sr	\$500	\$7.500
Integración Pagos	20	Senior	\$800	\$16.000
Panel Usuarios	12	Semi Sr	\$500	\$6.000
Migración BD	8	Senior	\$800	\$6.400
Costos adicionales				\$5.000
TOTAL				\$43.900

Análisis post-estimación (preguntas para defender cotización):

- ¿Función más costosa? Integración Pagos (\$16k). ¿Por qué? Senior + 20h + complejidad seguridad. ¿Optimizar? Usar API existente, Junior supervisado, reducir alcance.
- ¿Perfil más usado? Semi Senior + Senior (caro). ¿Eficiente? Sí si complejidad alta; si no, cambiar mix baja costo pero sube riesgo.
- ¿Costos extra inesperados? Licencias, alojamiento no estimados al inicio ? lección: checklist costos adicionales desde inicio.

13. Metodología de Sistemas de Información (10 Etapas)

Es la adaptación del enfoque básico de resolución de problemas (Qué hacer ? Cómo hacerlo ? Hacerlo ? Probar ? Usar) al ámbito SI, uniendo pensamiento lineal y sistémico.

Pasos básicos problema: Decidir Qué hacer ? Decidir Cómo hacerlo ? Hacerlo ? Probar resultado ? Usar producto.

10 Etapas (orden estricto para examen):

1) Reconocimiento

- Objetivo: Primer contacto con organización (tamaño, geografía, estructura formal/informal, cultura, necesidades). Establecer límites dónde trabajar, primera aproximación objetivos, planificar relevamiento. Conocer requerimientos/expectativas cliente, áreas afectadas, restricciones.
- Técnicas recolección: entrevistas, cuestionarios, observaciones + documentación negocio (misión, visión, valores, organigramas, manuales procedimientos, modelos procesos).
- Herramienta clave: Actas/Minutas reunión (documentan info recibida y validan con conforme de participantes + aportes).

2) Relevamiento

- Objetivo: Conocimiento exhaustivo base para desarrollo. Construir modelo/especificación de la realidad (no del sistema futuro), ahondando procesos, inconvenientes, cómo se perciben, causas, cómo superar.
- Tareas: 1) Recolección antecedentes (organización, falencias, cómo funciona SI actual y cómo desearían) 2) Análisis info 3) Modelado.
- Modelización cíclica: aproximaciones sucesivas validando con usuario, visión real no viciada por tecnología, para diagnosticar después.
- Técnicas: recolección (entrevistas predominantes, organigramas, glosarios) + documentación (cursogramas, modelos procesos). Siempre registrar, formalizar y consensuar con cliente.

3) Diagnóstico

- Objetivo: Determinar y formalizar problema real y sus causas (no solo síntoma reportado). Juicio objetivo, riguroso, eficaz para alternativas válidas.
- Base: Modelo construido en Relevamiento.
- Clave: El verdadero problema ? derivación directa síntoma usuario; debe consensuarse con cliente. Poner en palabras correctamente es primer paso a solución.

4) Estudio de Factibilidad

- Objetivo: Evaluar alternativas originadas en Diagnóstico y elegir la más viable según criterios consensuados.
- Principio: Equifinalidad (mismo destino por varios caminos) ? elige camino apropiado.
- Ponderación conjunta: Ing. SI + cliente (puede rechazar recomendación técnica por restricción/preferencia).
- Salida: Alternativa solución (puede combinar: compra/alquiler HW, compra/adaptación/desarrollo SW, capacitación, formalización procesos no informatizados, outsourcing, telecom).
- Cada componente genera proyectos asociados gestionados por Dirección SI. Para cada proyecto de desarrollo SW se invoca Ingeniería de Software.
- 4 Tipos de factibilidad:
- Técnica: ¿Implementable funcional/rendimiento/restricciones? ¿HW/SW tienen capacidades?
- Operativa: ¿Capacidad organizacional para cambio? ¿Personal tiene experiencia? ¿Capacitación requerida?
- Económica: ¿Relación costo/beneficio lo justifica?
- Legal: ¿Normativa vigente, aspectos políticos/legales?

5) Diseño

- Objetivo: Construir modelo de la solución usando requerimientos previos. Análisis y diseño datos y procesos, de conceptual a físico.
- Premisa: Cuanto mejor diseñado, más fácil implementar, mantener, transformar, con mayor confiabilidad y menor costo.

6) Desarrollo

- Objetivo: Construir lo diseñado. Generación código (programación) + convocatorias adquisición HW/SW, formalización procesos no informatizados, capacitación, customización SW adquirido, contratos locación, etc. Llevar a concreto lo propuesto.

7) Pruebas

- Objetivo: Encontrar mayor cantidad fallas, corregir, reducir índice error hacia estándares calidad. Previenen mayores desaciertos posteriores. Sin plan pruebas no hay garantía calidad. Permiten estimar, repetir, modificar validaciones posteriores.
- Estrategia: de lo específico a lo general (ver cap. 14):
- Unidad: cada unidad definida.
- Integración: relaciones entre módulos.
- Sistema: errores SI en entorno funcionamiento.
- Aceptación: participación usuario.
- Técnicas: Caja Negra (valida entradas vs salidas sin ver proceso) y Caja Blanca (valida y analiza pasos internos).

8) Implementación

- Objetivo: Instalar propuesta probada, desactivando productos previos.
- Desafío: Resistencia al cambio (respuesta frecuente). Requiere alto compromiso, adaptación paulatina.
- Estrategias: Funcionamiento paralelo temporal (viejo y nuevo) para ganar confianza; capacitación completa y apropiada para todos (a veces inicia etapas antes).

9) Mantenimiento

- Objetivo: Sostener solución viable en tiempo, asegurar validez. Calidad = costo cambio bajo.
- Pensamiento sistémico: solución no permanece inalterable, debe evolucionar con entorno y necesidades usuario.
- 3 Tipos acciones:
 - Correctivas: corrigen desvíos no detectados en Pruebas.
 - Perfectivas: perfeccionan/mejoran, nuevos requerimientos u optimizaciones rendimiento.
 - Adaptativas: cambios por usuario incorporando solución a rutina o por cambios tecnología/políticas/reglas (ej: cambio SO, nube, HW).
- Gestión configuraciones (gestión cambios): permite estimar correcciones y mejoras basadas en experiencia (costos, tiempos, horas hombre, recursos).
- Desafío cultural: generar conciencia que persistencia hace producto útil: robusto, íntegro, resistente al uso/desgaste. Calidad vinculada a mantenimiento, adaptabilidad y resistencia al cambio.

10) Sustitución

- Objetivo: Planificar salida/baja y reemplazo por nueva solución cuando mantenimiento resulta más costoso que cambio completo.
- Causas: económicos, tecnológicos, culturales, políticos, operativos o combinación.
- Clave: Planificar con mayor antelación posible, ejecución paulatina por etapas para evitar cortes abruptos, minimizar impacto usuarios, maximizar transición a mejores propuestas.

14. Pruebas

14.1 Estrategia General

Pruebas bajo nivel (pequeño segmento código) + alto nivel (funciones principales vs requisitos cliente).

Flujo:

Unidad (cada componente individual, técnicas rutas control) ? Integración (conflictos verificación/construcción, efecto adverso entre componentes, subfunciones no producen función principal, imprecisiones magnified) ? Validación (cumple requisitos informativos/funcionales/comportamiento/rendimiento) ? Sistema (mezcla HW, personal, BD, rendimiento global). Nota material integra validación dentro de sistema.

Objetivo testear: Encontrar errores con mínimo esfuerzo.

Características buena prueba:

1. Alta probabilidad encontrar error
2. No redundante (mismo propósito que otra)
3. "La mejor de la camada" (de grupo similar, elegir la que más clases errores descubre ? limitaciones tiempo/recursos)
4. No demasiado simple ni demasiado compleja

14.2 Caja Blanca (White Box)

Basada en examen detalles procedimiento, rutas lógicas y colaboraciones. Revisa conjuntos específicos condiciones/bucles.

Garantiza con casos prueba:

1. Todas rutas independientes dentro módulo revisadas ?1 vez
2. Todas decisiones lógicas en lado verdadero y falso
3. Todos bucles en fronteras y dentro fronteras operativas
4. Estructuras datos internas válidas

Prueba Ruta Básica (McCabe):

- Permite medir complejidad lógica (ciclomática) y derivar conjunto básico rutas que asegura ejecución todo enunciado ?1 vez.
- Paso 1: Grafo flujo control (cada estructura control tiene gráfico preestablecido: secuencia, condicional, mientras, hasta, selección múltiple).
 - Nodo (círculo): punto decisión/proceso
 - Arista (flecha): flujo
 - Región: área acotada por nodos/aristas; área fuera también es región
 - Condición compuesta: con OR/AND/NAND/NOR ? nodo separado por cada condición (ej: if A AND B ? 2 nodos predicado)

- Nodo predicado: contiene condición ? tiene ?2 aristas salientes
- Ruta independiente: introduce ?1 nuevo conjunto enunciados o nueva condición; debe recorrer ?1 arista no recorrida antes. No lo es si es combinación de ya especificadas sin arista nueva.
- Conjunto básico no es único. Método simplificado: selecciona camino más corto inicio-fin, luego busca segmentos no recorridos hasta completar nº requerido. Tiende a recorrer primero caminos excepción/errores.

Complejidad Ciclomática $V(G)$ - 3 FÓRMULAS EQUIVALENTES (MEMORIZAR):

1. $V(G)$ = Cantidad de Regiones
2. $V(G)$ = Cantidad Aristas - Cantidad Nodos + 2
3. $V(G)$ = Cantidad Nodos Predicados + 1

Significado: Número rutas independientes del conjunto básico = número mínimo de casos prueba para asegurar cobertura enunciados.

14.3 Caja Negra (Black Box)

Sobre interfaz, poca preocupación estructura interna, enfocada requisitos funcionales. Complementaria a blanca (descubre clases errores diferentes). Se aplica en etapas tardías.

Intenta encontrar:

1. Funciones incorrectas o faltantes
2. Errores interfaz
3. Errores estructuras datos / acceso BD externas
4. Errores comportamiento o rendimiento
5. Errores inicialización y terminación

Partición de Equivalencia: Divide dominio entrada en clases de las que derivan casos prueba. Basada en evaluación clases equivalencia para condición entrada.

4 Lineamientos para definir clases:

Condición Entrada | Clase Válida | Clases Inválidas

1. Especifica RANGO (ej: 1-100) | 1 clase dentro rango | 2 clases (por debajo y por encima)
2. Requiere VALOR ESPECÍFICO (ej: debe ser 5) | 1 clase con ese valor | 2 clases (menor y mayor)
3. Especifica MIEMBRO DE CONJUNTO (ej: debe ser "rojo", "verde", "azul") | 1 clase con miembro del conjunto | 1 clase fuera conjunto
4. Es BOOLEANA (ej: sí/no) | 1 clase | 1 clase

Cada clase válida/inválida genera al menos un caso prueba. Técnica reduce número casos manteniendo cobertura.

15. Anexo A: Tablas Resumen

Tabla 1: Modelos - Resumen 1 línea

- Cascada: Todo conocido, 1 entrega final, cliente ve al final.
- Incremental: Entregas funcionales parciales sin refinamiento.
- Espiral: Refinamiento + prototipos + gestión riesgo, iterativo.
- PU: Casos uso + arquitectura, iterativo-incremental, ejecutable cada iteración.
- Ágil: Sprints cortos, colaboración continua, responde al cambio.

Tabla 2: Riesgo - RMMM

Mitigación (evitar) ? Monitoreo (señales) ? Manejo/Contingencia (si sucede).

Tabla 3: Alcance - Incluido vs Futuro

Anotar siempre columna "Observaciones / Próxima versión" para negociar.

Tabla 4: Matriz Poder/Interés - Frase mnemotécnica

"Pocos Poderosos Satisfechos, Muchos Interesados Informados, Elites Gestionadas, Resto Monitoreado"

Tabla 5: Factibilidad 4 tipos

Técnica (¿Se puede construir?), Operativa (¿Se puede operar?), Económica (¿Conviene?), Legal (¿Se permite?).

Tabla 6: Mantenimiento 3 acciones

Correctiva (arregla lo que falló), Perfectiva (mejora lo que funciona), Adaptativa (adapta a contexto nuevo).

Tabla 7: Fórmulas Clave

- PERT: $VE = (O + 4M + P)/6$
- Costo: $(Esfuerzo \times Tarifa) + Adicionales$
- V(G): $Regiones = A - N + 2 = Predicados + 1$

16. Anexo B: Glosario

- Equifinalidad: Principio sistémico: mismo objetivo alcanzable por múltiples caminos ? justifica estudio factibilidad.
- Scope Creep (Expansión alcance): Crecimiento no controlado requisitos sin actualizar plan/costo.
- Backlog: Lista priorizada tareas pendientes (PO la gestiona en ágil).
- Sprint/Iteración: Ciclo corto tiempo fijo producción incremento.
- Outsourcing: Tercerización procedimientos.
- V(G): Complejidad ciclomática McCabe.
- Clase equivalencia: Conjunto valores entrada que se espera trate igual el sistema.
- Prueba regresión: Verificar cambios no introdujeron nuevas fallas.
- SQA: Aseguramiento calidad software (actividad sombrilla).
- Toxicidad equipo: Factores frenético/frustración/proceso fragmentado/roles difusos/fracaso repetido.
- Acta/Minuta: Documento validado por participantes que formaliza lo relevado.

17. Cómo Estudiar

Plan 3 pasadas:

1. Lectura rápida (1h): Solo tablas resumen + mapas. Marca lo que no entiendes.
2. Profunda (2h): Lee capítulos 2-14 con ejemplos, intenta explicar cada tabla en voz alta.
3. Activa (1h): Haz simulacro sin mirar, corrige con guía, repite fallos.

Tips parcial TUP:

- Si preguntan modelo para caso ? usa cuadro U1:25 + justifica con 2-3 factores (seguridad, feedback, recursos).
- Si piden calcular ? muestra fórmula, reemplazo, resultado y análisis (ej: "Función más costosa es X por... optimizaría con Y").
- Si piden stakeholder ? dibuja matriz 2x2 y ubica.
- Si piden riesgo ? presenta tabla 4 columnas + línea corte.
- Si piden prueba ? distingue blanca (estructura, V(G)) vs negra (interfaz, partición).

Guía generada a partir del material oficial P1. No reemplaza lectura de PDFs, pero cubre 100% de temas evaluables. ¡Éxitos!